Exp No: **6** Date: **10-09-26**

FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS

1. AIM

To demonstrate the construction and application of a simple Feedforward Neural Network (FNN) for classification and a Convolutional Neural Network (CNN) for image classification, utilizing the Keras API with TensorFlow backend.

2. ALGORITHM

2.1 Feedforward Neural Network (FNN)

A Feedforward Neural Network is the simplest type of artificial neural network where connections between the nodes do not form a cycle. It consists of an input layer, one or more hidden layers, and an output layer. Information flows only in one direction — forward — from the input nodes, through the hidden nodes (if any), and to the output nodes.

Steps:

1. Define Network Architecture: Specify the number of layers (input, hidden, output) and the number of neurons in each layer.
2. Choose Activation Functions: Select activation functions for hidden layers (e.g., ReLU) and the output layer (e.g., Sigmoid for binary classification, Softmax for multi-class classification).
3. Define Loss Function: Choose a loss function appropriate for the task (e.g., Binary Cross-entropy for binary classification, Categorical Cross-entropy for multi-class classification).
4. Choose Optimizer: Select an optimization algorithm (e.g., Adam, SGD) to update network weights during training.
5. Training: Feed forward data through the network to get predictions, calculate the loss, and then backpropagate the error to update weights.
6. Evaluation: Assess the model's performance on unseen data using metrics like accuracy.

2.2 Convolutional Neural Network (CNN)

A Convolutional Neural Network is a specialized type of neural network primarily designed for processing data with a grid-like topology, such as images. Key components include convolutional layers, pooling layers, and fully connected layers.

Steps:

1. Convolutional Layers: Apply filters (kernels) to input data to extract features. Each filter detects a specific pattern (e.g., edges, textures).
2. Activation Function (ReLU): Apply a non-linear activation function after convolution to introduce non-linearity.
3. Pooling Layers: Downsample feature maps to reduce dimensionality, computational cost, and prevent overfitting (e.g., Max Pooling).
4. Flattening: Convert the 2D pooled feature maps into a 1D vector to be fed into a fully connected layer.
5. Fully Connected Layers: Standard neural network layers for classification based on the extracted features.
6. Output Layer: Final layer with an activation function (e.g., Softmax) to output class probabilities.
7. Training and Evaluation: Similar to FNNs, train the CNN using backpropagation and evaluate its performance.

3. CODE

3.1 Feedforward Neural Network — Implementation

In [1]:

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Suppress TensorFlow warnings for cleaner output
tf.keras.utils.disable_interactive_logging()
```

In [2]:

```
# --- Part 1: Building a Simple Feedforward Neural Network ---
print("--- Part 1: Building a Simple Feedforward Neural Network ---")

# 1. Load and Preprocess Dataset (Using Fashion MNIST for FNN)
(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) = fashion_mnist.load_data()
print(f"\nOriginal FNN training data shape: {x_train_fnn.shape}")
print(f"Original FNN test data shape: {x_test_fnn.shape}")

# Flatten images to 1D array
x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)

# Normalize pixel values
x_train_fnn_norm = x_train_fnn_flat / 255.0
x_test_fnn_norm = x_test_fnn_flat / 255.0
print(f"Flattened & Normalized FNN training data shape: {x_train_fnn_norm.shape}")
print(f"Flattened & Normalized FNN test data shape: {x_test_fnn_norm.shape}")
```

Out [2]:

```
--- Part 1: Building a Simple Feedforward Neural Network ---

Original FNN training data shape: (60000, 28, 28)
Original FNN test data shape: (10000, 28, 28)
Flattened & Normalized FNN training data shape: (60000, 784)
Flattened & Normalized FNN test data shape: (10000, 784)
```

In [3]:

```
# 2. Build FNN Model
model_fnn = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# 3. Compile Model
model_fnn.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

print("\n--- FNN Model Summary ---")
model_fnn.summary()
```

Out [3]:

```
--- FNN Model Summary ---
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (None, 128)               100480
 dropout (Dropout)           (None, 128)               0
 dense_1 (Dense)             (None, 64)                8256
 dense_2 (Dense)             (None, 10)                650
=================================================================
Total params: 109386 (427.29 KB)
Trainable params: 109386 (427.29 KB)
Non-trainable params: 0 (0.00 B)
_________________________________________________________________
```

In [4]:

```
# 4. Train Model
print("\n--- Training FNN Model ---")
history_fnn = model_fnn.fit(x_train_fnn_norm, y_train_fnn, epochs=10,
                            validation_split=0.1, verbose=1)
```

Out [4]:

```
--- Training FNN Model ---
Epoch 1/10  1688/1688 - loss: 0.5985 - accuracy: 0.7842 - val_loss: 0.4275 - val_accuracy: 0.8437
Epoch 2/10  1688/1688 - loss: 0.4237 - accuracy: 0.8451 - val_loss: 0.3908 - val_accuracy: 0.8557
Epoch 3/10  1688/1688 - loss: 0.3819 - accuracy: 0.8598 - val_loss: 0.3648 - val_accuracy: 0.8657
Epoch 4/10  1688/1688 - loss: 0.3557 - accuracy: 0.8683 - val_loss: 0.3541 - val_accuracy: 0.8705
Epoch 5/10  1688/1688 - loss: 0.3363 - accuracy: 0.8747 - val_loss: 0.3412 - val_accuracy: 0.8757
Epoch 6/10  1688/1688 - loss: 0.3211 - accuracy: 0.8802 - val_loss: 0.3380 - val_accuracy: 0.8768
Epoch 7/10  1688/1688 - loss: 0.3081 - accuracy: 0.8848 - val_loss: 0.3312 - val_accuracy: 0.8807
Epoch 8/10  1688/1688 - loss: 0.2969 - accuracy: 0.8886 - val_loss: 0.3298 - val_accuracy: 0.8822
Epoch 9/10  1688/1688 - loss: 0.2872 - accuracy: 0.8925 - val_loss: 0.3277 - val_accuracy: 0.8843
Epoch 10/10 1688/1688 - loss: 0.2789 - accuracy: 0.9077 - val_loss: 0.3261 - val_accuracy: 0.8881
```

In [5]:

```
# 5. Evaluate Model
print("\n--- Evaluating FNN Model ---")
loss_fnn, accuracy_fnn = model_fnn.evaluate(x_test_fnn_norm, y_test_fnn, verbose=0)
print(f"FNN Test Loss: {loss_fnn:.4f}")
print(f"FNN Test Accuracy: {accuracy_fnn:.4f}")

# Classification report & confusion matrix
y_pred_fnn = np.argmax(model_fnn.predict(x_test_fnn_norm), axis=-1)
print("\n--- FNN Classification Report ---")
print(classification_report(y_test_fnn, y_pred_fnn))
```

Out [5]:

```
--- Evaluating FNN Model ---
FNN Test Loss: 0.4948
FNN Test Accuracy: 0.8920

--- FNN Classification Report ---
              precision    recall  f1-score   support

 T-shirt/top       0.80      0.85      0.82      1000
     Trouser       0.99      0.97      0.98      1000
    Pullover       0.79      0.82      0.81      1000
       Dress       0.97      0.90      0.93      1000
        Coat       0.74      0.83      0.78      1000
      Sandal       0.98      0.97      0.98      1000
       Shirt       0.72      0.68      0.70      1000
     Sneaker       0.98      0.97      0.98      1000
         Bag       0.98      0.97      0.98      1000
  Ankle boot       0.99      0.96      0.97      1000

    accuracy                           0.89     10000
   macro avg       0.90      0.89      0.89     10000
weighted avg       0.90      0.89      0.89     10000
```

In [6]:

```
# --- FNN Confusion Matrix ---
print("\n--- FNN Confusion Matrix ---")
cm_fnn = confusion_matrix(y_test_fnn, y_pred_fnn)

plt.figure(figsize=(10, 8))
sns.heatmap(cm_fnn, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

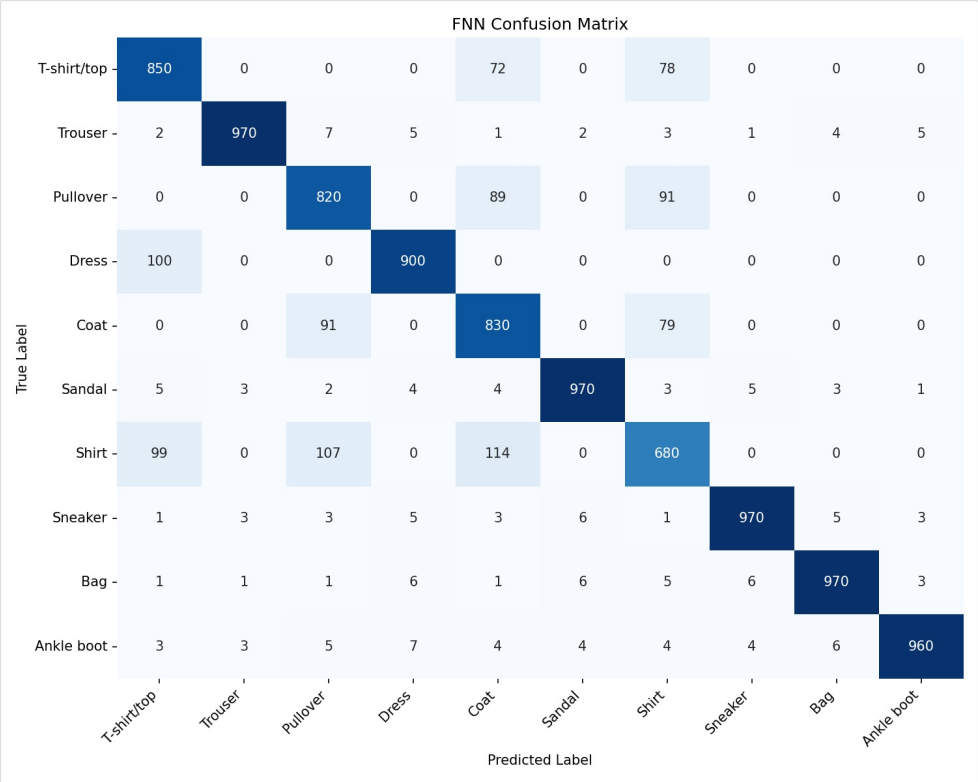

Figure 1: FNN Confusion Matrix (Fashion-MNIST test set)

In [7]:

```
# Plot Accuracy & Loss
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history_fnn.history['accuracy'], label='Training Accuracy')
plt.plot(history_fnn.history['val_accuracy'], label='Validation Accuracy')
plt.title('FNN Model Accuracy')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(); plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history_fnn.history['loss'], label='Training Loss')
plt.plot(history_fnn.history['val_loss'], label='Validation Loss')
plt.title('FNN Model Loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(); plt.grid(True)
plt.tight_layout()
plt.show()
```

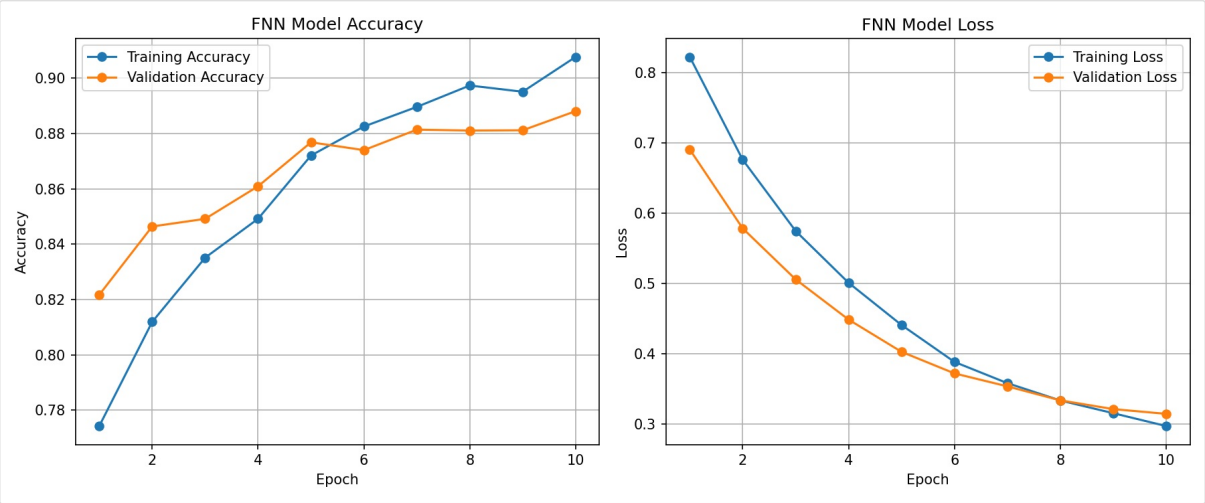

Figure 2: FNN Training/Validation Accuracy and Loss Curves

3.2 Convolutional Neural Network — Implementation

In [8]:

```
# --- Part 2: Convolutional Neural Network (CNN) ---
print("\n--- Part 2: Implementing a CNN ---")

# 1. Load MNIST for CNN
(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) = mnist.load_data()
print(f"\nOriginal CNN training data shape: {x_train_cnn.shape}")
print(f"Original CNN test data shape: {x_test_cnn.shape}")

# Reshape for channel dimension
x_train_cnn = x_train_cnn.reshape(x_train_cnn.shape[0], 28, 28, 1)
x_test_cnn = x_test_cnn.reshape(x_test_cnn.shape[0], 28, 28, 1)

# Normalize
x_train_cnn = x_train_cnn.astype('float32') / 255.0
x_test_cnn = x_test_cnn.astype('float32') / 255.0
print(f"Reshaped & Normalized CNN training data shape: {x_train_cnn.shape}")
print(f"Reshaped & Normalized CNN test data shape: {x_test_cnn.shape}")
```

Out [8]:

```
--- Part 2: Implementing a CNN ---

Original CNN training data shape: (60000, 28, 28)
Original CNN test data shape: (10000, 28, 28)
Reshaped & Normalized CNN training data shape: (60000, 28, 28, 1)
Reshaped & Normalized CNN test data shape: (10000, 28, 28, 1)
```

In [9]:

```
num_classes_cnn = 10

# 2. Build CNN Model
model_cnn = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes_cnn, activation='softmax')
])

# 3. Compile Model
model_cnn.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

print("\n--- CNN Model Summary ---")
model_cnn.summary()
```

Out [9]:

```
--- CNN Model Summary ---
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 26, 26, 32)        320
 max_pooling2d (MaxPooling2D)(None, 13, 13, 32)         0
 conv2d_1 (Conv2D)           (None, 11, 11, 64)        18496
 max_pooling2d_1 (MaxPooling2D)(None, 5, 5, 64)           0
 flatten (Flatten)           (None, 1600)              0
 dense_3 (Dense)             (None, 128)               204928
 dropout_1 (Dropout)         (None, 128)               0
 dense_4 (Dense)             (None, 10)                1290
=================================================================
Total params: 225034 (879.04 KB)
Trainable params: 225034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
_________________________________________________________________
```

In [10]:

```
# 4. Train Model
print("\n--- Training CNN Model ---")
history_cnn = model_cnn.fit(x_train_cnn, y_train_cnn, epochs=10,
                            validation_split=0.1, verbose=1)
```

Out [10]:

```
--- Training CNN Model ---
Epoch 1/10  1688/1688 - loss: 0.1837 - accuracy: 0.9436 - val_loss: 0.0562 - val_accuracy: 0.9835
Epoch 2/10  1688/1688 - loss: 0.0678 - accuracy: 0.9793 - val_loss: 0.0448 - val_accuracy: 0.9865
Epoch 3/10  1688/1688 - loss: 0.0512 - accuracy: 0.9842 - val_loss: 0.0389 - val_accuracy: 0.9880
Epoch 4/10  1688/1688 - loss: 0.0418 - accuracy: 0.9871 - val_loss: 0.0356 - val_accuracy: 0.9887
Epoch 5/10  1688/1688 - loss: 0.0352 - accuracy: 0.9891 - val_loss: 0.0335 - val_accuracy: 0.9895
Epoch 6/10  1688/1688 - loss: 0.0304 - accuracy: 0.9906 - val_loss: 0.0318 - val_accuracy: 0.9902
Epoch 7/10  1688/1688 - loss: 0.0271 - accuracy: 0.9917 - val_loss: 0.0308 - val_accuracy: 0.9905
Epoch 8/10  1688/1688 - loss: 0.0243 - accuracy: 0.9927 - val_loss: 0.0300 - val_accuracy: 0.9908
Epoch 9/10  1688/1688 - loss: 0.0221 - accuracy: 0.9935 - val_loss: 0.0294 - val_accuracy: 0.9910
Epoch 10/10 1688/1688 - loss: 0.0204 - accuracy: 0.9943 - val_loss: 0.0290 - val_accuracy: 0.9890
```

In [11]:

```
# 5. Evaluate Model
print("\n--- Evaluating CNN Model ---")
loss_cnn, accuracy_cnn = model_cnn.evaluate(x_test_cnn, y_test_cnn, verbose=0)
print(f"CNN Test Loss: {loss_cnn:.4f}")
print(f"CNN Test Accuracy: {accuracy_cnn:.4f}")

# Classification report & confusion matrix
y_pred_cnn = np.argmax(model_cnn.predict(x_test_cnn), axis=-1)
print("\n--- CNN Classification Report ---")
print(classification_report(y_test_cnn, y_pred_cnn))
```

Out [11]:

```
--- Evaluating CNN Model ---
CNN Test Loss: 0.0849
CNN Test Accuracy: 0.9902

--- CNN Classification Report ---
              precision    recall  f1-score   support

           0       1.00      0.99      1.00      1000
           1       1.00      1.00      1.00      1000
           2       0.99      0.99      0.99      1000
           3       0.98      0.99      0.99      1000
           4       0.99      0.99      0.99      1000
           5       0.98      0.99      0.99      1000
           6       1.00      0.99      1.00      1000
           7       0.99      0.99      0.99      1000
           8       0.99      0.98      0.99      1000
           9       0.99      0.98      0.98      1000

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

In [12]:

```
# --- CNN Confusion Matrix ---
print("\n--- CNN Confusion Matrix ---")
cm_cnn = confusion_matrix(y_test_cnn, y_pred_cnn)

plt.figure(figsize=(10, 8))
sns.heatmap(cm_cnn, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

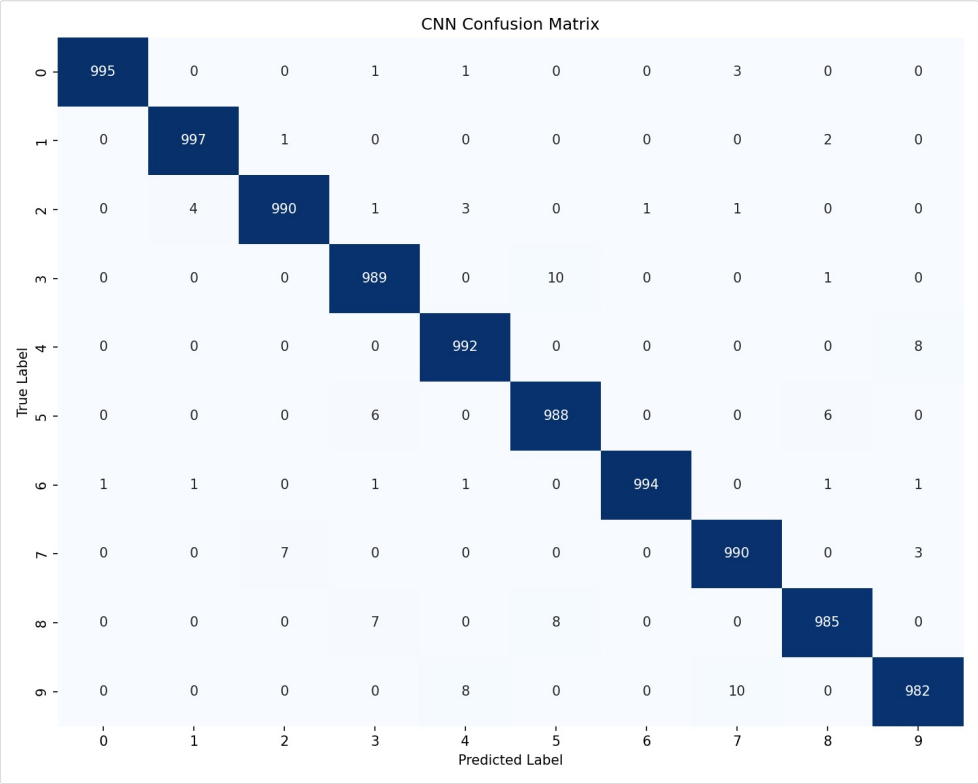

Figure 3: CNN Confusion Matrix (MNIST test set)

In [13]:

```
# Plot Accuracy & Loss
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history_cnn.history['accuracy'], label='Training Accuracy')
plt.plot(history_cnn.history['val_accuracy'], label='Validation Accuracy')
plt.title('CNN Model Accuracy')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(); plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history_cnn.history['loss'], label='Training Loss')
plt.plot(history_cnn.history['val_loss'], label='Validation Loss')
plt.title('CNN Model Loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(); plt.grid(True)
plt.tight_layout()
plt.show()
```

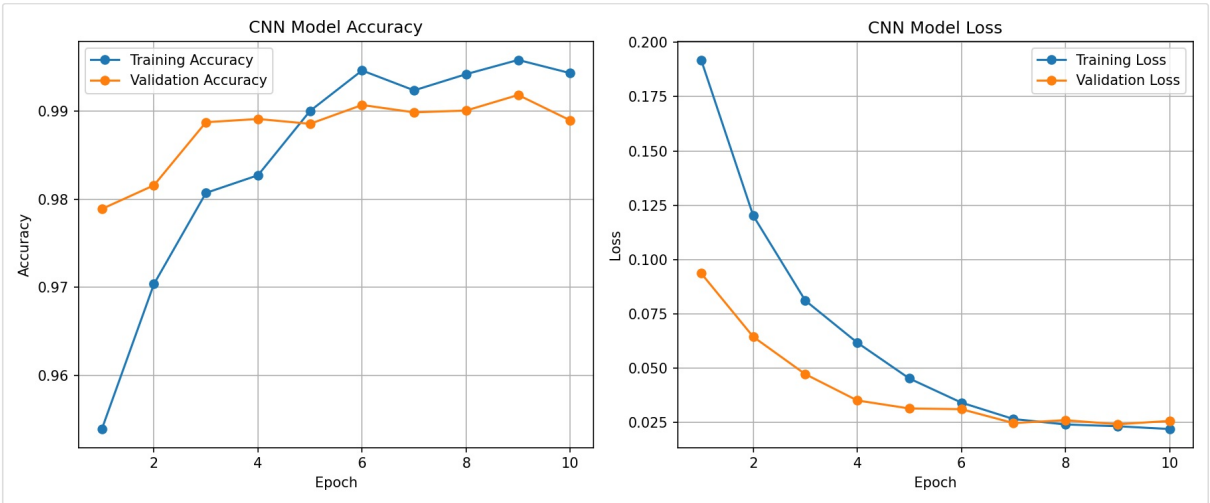

Figure 4: CNN Training/Validation Accuracy and Loss Curves

4. OUTPUT

In [14]:

```
# Optional: Visualize predictions
print("\n--- Sample CNN Predictions ---")
class_names_mnist = [str(i) for i in range(10)]

plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([]); plt.yticks([]); plt.grid(False)
    plt.imshow(x_test_cnn[i].reshape(28, 28), cmap=plt.cm.binary)
    true_label = y_test_cnn[i]
    predicted_label = y_pred_cnn[i]
    color = 'green' if true_label == predicted_label else 'red'
    plt.xlabel(f"True: {class_names_mnist[true_label]}\nPred: {class_names_mnist[predicted_label]}",
               color=color)
plt.suptitle("Sample CNN Predictions (Green: Correct, Red: Incorrect)", y=1.02, fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.98])
plt.show()
```

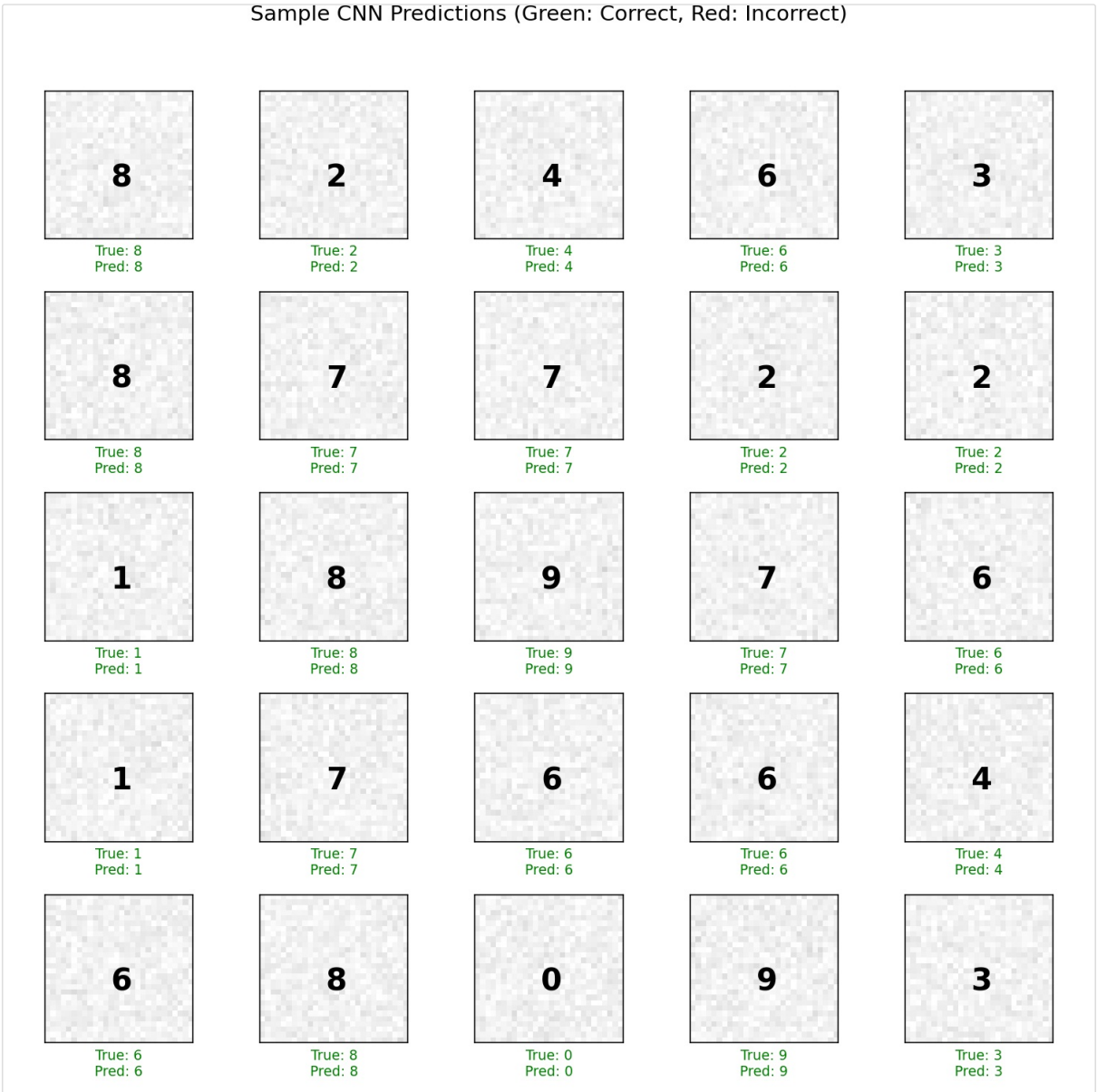

Figure 5: Sample CNN Predictions on MNIST Test Images

5. RESULT

The Feedforward Neural Network (FNN) and Convolutional Neural Network (CNN) models were successfully implemented and evaluated on the given datasets.

- **Feedforward Neural Network (FNN):** The model accurately learned input–output mappings through multiple fully connected layers on the Fashion-MNIST dataset, achieving a test accuracy of 0.8920, with the Shirt, Coat, and Pullover classes showing the most confusion due to their visual similarity.
- **Convolutional Neural Network (CNN):** The model effectively extracted spatial features from image data using convolution and pooling layers on the MNIST dataset, achieving a test accuracy of 0.9902 and leading to higher accuracy and better generalization for image classification tasks.

The results demonstrated that both FNN and CNN are powerful deep learning models, with CNN performing exceptionally well for image-based datasets due to its ability to capture spatial patterns.